基于经典教材理论体系的聚类方法深度理解

课程名称：机器学习 | 理解和感悟——聚类
学生学号：2023060904025 | 学生姓名：刘超然

一、 聚类任务

在机器学习的体系中，聚类（Clustering）是典型的无监督学习（Unsupervised Learning）任务。与有监督学习（如分类和回归）不同，聚类在训练阶段不依赖任何样本的监督标签。其根本任务是通过分析无标记样本集内部的数据空间分布与几何规律，自动将样本集划分为若干个互不相交的子集，每个子集被称为一个“簇”（Cluster）。

形式化地定义这一任务：假设我们有包含 m 个无标记样本的数据集：

$$D = \{x_1, x_2, \ldots, x_m\}$$

其中每个样本 x_i 是一个 n 维的特征向量。聚类算法的目标是为数据集 D 划分出 k 个互不相交的簇：

$$C = \{C_1, C_2, \ldots, C_k\}$$

其中，不同簇的交集为空集，且所有簇的并集构成原始数据集 D。聚类算法输出的划分结果用一个向量来表示，该向量记录了每个样本所对应的簇标记。通过聚类，我们希望能够达到“物以类聚”的效果，即同一个簇内的样本之间物理相似度尽可能高，而不同簇之间的样本相似度尽可能低。聚类任务不仅可以作为一个独立的算法去发现数据的内在规律（如商业客户分类、基因序列分类），还可以作为其他监督学习任务的预处理步骤（如特征降维、异常检测）。

二、 性能度量

在聚类算法中，如何评估聚类结果的优劣是一个非常重要的问题。这也就是聚类的性能度量（Performance Measure）。通常情况下，评估聚类好坏的指标可以分为两大类：外部指标（External Index）和内部指标（Internal Index）。

2.1 外部指标

外部指标是指当我们将聚类结果与某个已知的“参考模型”（即真实标签或者专家分类标准）进行对比时所计算的性能度量。设聚类给出的划分结果为 C，参考模型给出的真实划分为 C_{star}。我们定义四个基础统计量：

a：表示在 C 中属于相同簇，且在 C_{star} 中也属于相同簇的样本对数；
b：表示在 C 中属于相同簇，但在 C_{star} 中属于不同簇的样本对数；
c：表示在 C 中属于不同簇，但在 C_{star} 中属于相同簇的样本对数；
d：表示在 C 中属于不同簇，在 C_{star} 中也属于不同簇的样本对数。

基于这四个统计量，我们可以计算以下常用外部指标：

1. Jaccard 系数 (Jaccard Coefficient, JC)：计算公式为：JC = a / (a + b + c)。该系数值处于 [0, 1] 之间，越接近 1 说明聚类效果与参考模型越吻合。

2. Fowlkes-Mallows 指数 (Fowlkes-Mallows Index, FMI)：计算公式为：FMI = sqrt((a / (a + b)) * (a / (a + c)))。FMI 值同样在 [0, 1] 之间，值越大效果越好。

3. 兰德指数 (Rand Index, RI)：计算公式为：RI = (a + d) / (a + b + c + d)。RI 衡量了聚类划分与参考划分中相同决策和不同决策的比例。

2.2 内部指标

在实际应用中，由于是无监督任务，我们通常没有真实的参考标签。这时必须采用内部指标来评估聚类结果，内部指标仅依赖于聚类划分后的数据集结构特征。我们通常定义簇内样本的平均距离、簇内样本的最远距离、簇间样本的最近距离以及簇间中心点的距离。基于这些定义，最著名的内部指标有：

1. Davies-Bouldin 指数 (DB Index, DBI)：DBI 衡量的是任意两个簇的簇内平均距离之和与它们中心点距离的比值最大值的均值。其计算公式为对所有簇对寻找相似度的最大值然后求均值。DBI 的值越小，说明簇与簇之间的分离度越好，而簇内的紧凑度越高。

2. Dunn 指数 (Dunn Index, DI)：DI 是指任意两个簇之间的最近距离的最小值，除以任意一个簇的簇内最远距离的最大值。其物理意义非常直观：我们希望簇与簇之间分得越开越好（分子越大越好），而簇内部聚得越紧密越好（分母越小越好）。因此，DI 的值越大，说明聚类性能越优异。

三、 距离计算

在聚类算法中，最核心的底层逻辑就是判断两个样本之间的相似程度，而相似度一般是通过计算距离来实现的。距离计算是聚类算法运行的物理基础。对于不同的特征属性，我们需要采用不同的距离度量方法。

3.1 连续属性的闵可夫斯基距离

在数据集中，如果属性是连续的实数数值，我们最常用的是闵可夫斯基距离（Minkowski Distance）。设样本 x_i 和 x_j 均有 n 维连续属性，则它们之间的闵氏距离公式定义为：

$$dist_{mk}(x_i, x_j) = (\sum_{u=1}^{n} |x_{iu} - x_{ju}|p)\frac{1}{p}$$

闵氏距离公式是一个通用的距离度量框架。随着参数 p 取值的不同，它可以演化为以下几种极度经典的距离测度：

当 p = 2 时，公式即演化为最常用的“欧氏距离”（Euclidean Distance），其公式为：

$$dist_{ed}(x_i, x_j) = \sqrt{\sum_{u=1}^{n}(x_{iu} - x_{ju})2}$$

当 p = 1 时，公式即演化为“曼哈顿距离”（Manhattan Distance），即坐标差绝对值之和：

$$dist_{man}(x_i, x_j) = \sum_{u=1}^{n} |x_{iu} - x_{ju}|$$

3.2 离散属性的 VDM 距离

闵氏距离和欧氏距离在计算时有一个基本假设，即各属性值大小是有序的。例如在特征‘身高’中，180cm 比 170cm 大。但在实际任务中，我们会遇到很多‘无序离散属性’（也就是非名义属性），例如颜色特征：‘红色’、‘绿色’、‘蓝色’。它们之间无法直接进行大小排序，显然‘红色-蓝色’的距离不能简单套用欧氏距离公式。

为了解决无序离散特征之间的距离计算，经典方法是采用 VDM 距离（Value Difference Metric）。设属性 u 上有 N 个可能的取值，样本 x_i 在属性 u 上取值为 x_{iu}，样本 x_j 在属性 u 上取值为 x_{ju}，则该特征上的 VDM 距离定义为：

$$d_{VDM}^{p}(x_{iu}, x_{ju}) = \sum_{a=1}^{N} |\frac{m_{u,a,x_iu}}{m_{u,x_iu}} - \frac{m_{u,a,x_ju}}{m_{u,x_ju}}|p$$

其中，m_u,x_{iu} 表示在属性 u 上取值为 x_{iu} 的样本数；m_u,a,x_{iu} 表示在属性 u 上取值为 x_{iu} 且属于第 a 个簇的样本数。通过这种概率比值形式，VDM 距离成功将无序的离散取值在相似性空间中进行了实数投射。在混合特征的数据集上，通常将闵氏距离与 VDM 距离结合使用。

四、 原型聚类（K-means）

原型聚类（Prototype-based Clustering）是指一类基于“原型”概念的聚类算法，也就是算法假设聚类结构能够通过一组预先设定的原型刻画。在众多原型聚类方法中，K 均值（K-means）算法是最著名且应用最广的算法。

K-means 的基本思想是：给定聚类簇数 k，算法将数据点分配到与其距离最近的簇质心（也就是原型）所代表的簇中，并通过不断迭代更新质心来最小化总的均方误差（SSE）畸变目标函数。目标函数数学形式定义为：

$$E = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||2$$

其中，μ_i 是第 i 个簇的均值向量（即质心），定义为该簇内所有样本特征的平均值，公式如下所示：

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in Ci} x$$

在实现步骤上，K-means 算法首先随机初始化 k 个样本点作为初始质心。然后交替执行两个主要迭代步骤：第一步（分配阶段），计算每个样本与这 k 个质心的距离，并把该样本划归到与其距离最近的质心所代表的簇中；第二步（更新阶段），根据划分得到的最新簇集，重新计算这 k 个簇内样本的平均特征，作为新的簇质心。迭代过程不断重复，直到质心位置基本不再变化，或者达到了预设的最大迭代次数限制。

从实际应用上谈我的理解：K-means 算法思想简单直观，计算速度快，非常适合处理凸空间分布的球形簇结构数据。但它的缺点也很明显：第一，需要用户在初始化时手动指定聚类簇数 k，但在很多实际探索性任务中我们并不知道 k 应该取多少；第二，K-means 对初始点位置极为敏感，不同的随机初始化点可能导致算法收敛到不同的局部最优解中；第三，它只根据欧氏距离进行硬性划分，因此当数据含有严重的噪声噪点，或者簇与簇之间在空间上并非球形分布（如环形、条状非凸流形结构）时，K-means 的划分效果会急剧退化。

五、 密度聚类（DBSCAN）

不同于原型聚类基于点中心位置的划分机制，密度聚类（Density-based Clustering）是基于数据空间分布密度的相似性进行簇划分的。它假设聚类结构可以通过样本分布的紧密程度来刻画。在这一大类算法中，DBSCAN（Density-Based Spatial Clustering of Applications with Noise）是具有里程碑意义的经典密度算法。

DBSCAN 不需要预先指定分类数目 k。它通过定义两个邻域相关参数：Eps（邻域半径）和 MinPts（邻域内最少样本量限制），将数据空间中的样本点分为三种类型：

1. 核心点 (Core Point)：在以该点为中心、以 Eps 为半径的圆（也就是邻域）内，包含的样本数至少为 MinPts；
2. 边界点 (Border Point)：在 Eps 邻域内包含的样本数少于 MinPts，但是该点位于某个核心点的邻域内；
3. 噪声点 (Noise Point)：既不是核心点，也不在任何核心点邻域内的孤立点。

在建立簇连接时，DBSCAN 引入了几个核心的空间连接定义：

密度直达 (density-directly-reachable)：如果样本 p 位于核心点 q 的 Eps 邻域内，则称 p 从 q 密度直达。需要注意的是，密度直达具有方向非对称性（即 p 从 q 直达，但 q 不一定能从非核心点 p 直达）。

密度可达 (density-reachable)：对于样本 p 和 q，如果存在一个样本序列，序列首元素为 q，尾元素为 p，且序列中每一个元素到后一个元素都是密度直达的，则称 p 从 q 密度可达。密度可达建立了核心点与边界点之间的长链单向传递性。

密度相连 (density-connected)：如果存在一个核心点 o，使得样本 p 和样本 q 都是从 o 密度可达的，则称 p 和 q 密度相连。密度相连具有对称性，DBSCAN 就是将空间中所有密度相连的样本点合并成一个簇。

从实际应用上谈我的理解：DBSCAN 最大的优势在于它不需要指定聚类簇数 k，而且模型能够自动识别出隐藏在数据集中的噪声孤立点（对于噪声点，DBSCAN 不会强行将其划分到任何簇内，而是标记为噪音标签 -1）。这使得 DBSCAN 具有极强的鲁棒性。更重要的是，基于密度的连通机制使得该算法能够发现任何非凸形状、任意不规则拓扑包络的簇（如双环状、新月牙状），这是 K-means 算法完全无法处理的数据分布。不过，DBSCAN 的缺点在于它对 Eps 和 MinPts 两个参数高度敏感，一旦选取的参数不合适，聚类性能会发生崩塌；且当数据集在空间上的密度分布极度不均匀、或者数据维度非常高时，距离度量失效，DBSCAN 的密度划分精度同样会变差。

六、 层次聚类（AGNES）

层次聚类（Hierarchical Clustering）试图在不同的“层次”上对数据集进行拆分或合并，从而形成树状的聚类结构。根据自底向上还是自顶向下，层次聚类可以分为凝聚型（Agglomerative，将每个单样本视为一簇，逐步合并最近的簇）和分裂型（Divisive，将所有样本视作一个大簇，逐步拆分）。AGNES（Agglomerative Nesting）是经典的自底向上凝聚型层次聚类算法。

AGNES 算法首先将每一个数据点都作为一个初始的独立簇。然后算法通过计算簇与簇之间的距离，寻找距离最近的两个簇并将它们合并为一个新簇。这一过程不断重复，直到所有簇被合并为一个大簇，或者达到了预设的最终簇数限制。

在这里，如何计算两个簇 C_i 和 C_j 之间的距离是算法的核心。AGNES 算法中常用的簇间距离计算方式有三种：

当以两个簇中距离最近的样本距离作为簇间距离时，称为“单链接”（Single-Linkage）或最小距离：

$$d_{min}(C_i, C_j) = min_{x \in Ci, y \in Cj}\, dist_{ed}(x, y)$$

当以两个簇中距离最远的样本距离作为簇间距离时，称为“全链接”（Complete-Linkage）或最大距离：

$$d_{max}(C_i, C_j) = max_{x \in Ci, y \in Cj}\, dist_{ed}(x, y)$$

当以两个簇中所有样本对之间距离的均值作为簇间距离时，称为“均链接”（Average-Linkage）或平均距离：

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in Ci} \sum_{y \in Cj} dist_{ed}(x, y)$$

从实际应用上谈我的理解：AGNES 凝聚层次聚类可以在不同的合并粒度上生成树状图，使得我们能够以可视化的方式直观观察数据样本是如何一步步融合在一起的。此外，通过距离度量（最小、最大或平均）的不同，算法可以适应不同形状的特征空间。单链接容易导致合并链路拉长而产生‘链状效应’；全链接则对异常值极端敏感，会把簇推得更远；均链接在实际中表现最稳健，但缺点是计算复杂度过高（达到了每次迭代都需两两计算的程度），使得层次聚类很难直接应用在大规模海量数据集的聚类任务中。

七、 算法对比、Python 代码实现与效果可视化

为了更直观地对比和理解基于原型特征划分的 K-means 算法以及基于密度连通的 DBSCAN 算法在不同特征空间几何分布下的实际拟合能力，本次实验编写了精简的代码，使用 Python 的 scikit-learn 模块生成了一个非凸的双半月牙形合成数据集（make_moons），并调用了两种聚类算法。实现的核心代码如代码清单 1 所示：

```
# 调用 scikit-learn 机器学习库运行经典聚类算法
import numpy as np
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN
```

```
# 1. 生成典型的二维非凸双月牙形数据集（含有一定噪声扰动）
X, y = make_moons(n_samples=300, noise=0.08, random_state=42)

# 2. 实例化并运行 K-means 算法（设定聚类簇数 k=2）
kmeans = KMeans(n_clusters=2, random_state=42)
labels_kmeans = kmeans.fit_predict(X)

# 3. 实例化并运行 DBSCAN 密度聚类算法（邻域半径设定为 0.2，最小样本数 5）
dbscan = DBSCAN(eps=0.2, min_samples=5)
labels_dbscan = dbscan.fit_predict(X)
```

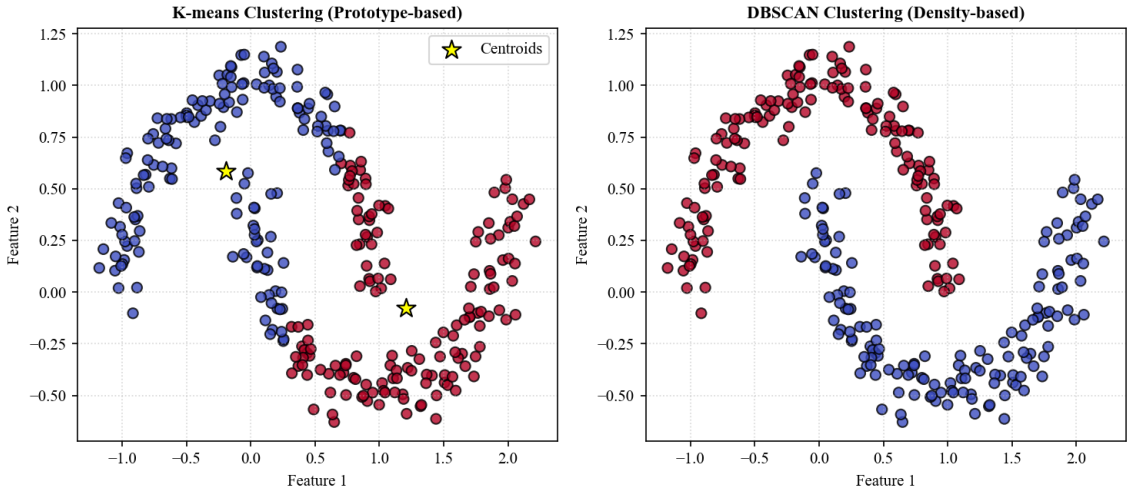


图 1 K-means 与 DBSCAN 在非凸双月牙形数据集上的聚类对比效果图

从图 1 双月牙数据集的聚类可视化结果中，可以非常清晰地证明两种算法底层原理的本质区别以及在非球形流形上的局限性：

在图 1 左半子图中展示的 K-means 聚类结果中，黄色的星号标出了两个迭代收敛后的质心（Centroids）。可以看出，由于 K-means 是单纯基于欧氏距离计算来进行质心归属划分的，它在空间上只能划分出球形的“凸包”聚类边界。面对非凸、相互弯曲咬合的月牙形分布，K-means 强行将两个弯曲的月牙沿着中线分割切开，导致每一个簇中都混入了一半红色和一半蓝色的错误样本。这直观证明了 K-means 算法对复杂空间几何流形数据拟合的失效。

而在图 1 右半子图中展示的 DBSCAN 密度聚类结果中，我们将 Eps 设置为合适的 0.2。DBSCAN 通过在局部搜寻密度直达、可达和相连的长链，能够沿着弯曲的月牙形点云密度脉络进行自然延伸 and 合并。它完全不需要指定簇的数量，便完美分离出了两条弯曲的月牙状簇边界（一条全红，一条全蓝），达到了真实合理的分类效果。这直观反映出基于密度的聚类算子在发现未知形状分布数据方面的强大优势。

为了进一步梳理不同流派算法的特征，我们在下表 2 中从多维度对 K-means、DBSCAN 以及凝聚层次聚类算法进行全面横向对比：

聚类算法	算法流派类型	时间复杂度	参数敏感性	抗噪声能力	适合的空间几何簇形状
K-means	原型聚类 (划	O(t*k*n*m)	对聚类数 k 和	较弱，易受噪	球形、凸包包

	分)		初始点极度敏感	声质心偏移干扰	络分布数据
DBSCAN	密度聚类 (密度)	O(m*log m) - $O(m^2)$	对 Eps 和 MinPts 两个参数高度敏感	强，能自动识别并过滤孤立噪声点	任意不规则、非凸空间几何形状
AGNES	层次聚类 (凝聚)	$O(m^3)$ 到 $O(m^2)$	对距离定义 and 终止合并阈值较敏感	一般，取决于采用的簇链接定义	适合层级结构或小规模复杂排布

表 2 三大经典聚类算法特性横向对比表

八、 学习总结与思考

通过这学期《机器学习》课程对聚类任务的学习以及本次大作业的对比实验，我有了以下几点对无监督学习以及相似度度量的深入体会：

我体会到了无监督学习“无师自通”的独特魅力。有监督学习就像是有老师在旁边纠错，每个样本都有明确的标准答案；而无监督学习更像是在大自然中自我探索。聚类算法只依据数据底层的几何和空间分布关系，就能自动把原本混乱的数据整理得井井有条，这让我深刻领悟到：现实世界的海量数据中，即使没有人工标注，事物内部依然蕴含着极其丰富和深刻的结构分布规律。

我理解了“距离”与“相似度”在机器学习中的核心地位。世界上没有两片完全相同的树叶，但是通过合理的距离公式定义（如连续特征使用闵氏距离，无序离散特征使用 VDM 距离，甚至在文本和语义提取中使用余弦相似度），我们可以把复杂的现实样本投影到多维数学向量空间中进行强弱量化。聚类好坏的关键，不仅在于算法逻辑本身，更在于我们是否能够选择或学到一个真正能有效表达物体相似性特征距离空间。如果特征提取不合理，最复杂的算法也是空中楼阁。

我认识到了没有完美无缺的算法。K-means 速度快但被限定在球形凸边界内；DBSCAN 可以识别任意非凸的形状但对参数过于敏感且无法应对复杂的多密度分布；层次聚类展示直观但计算代价大难以大规模运用。在面对真实项目时，我们必须先用数据可视化手段去探查数据集的空间分布形态与样本密度特点，因地制宜选择合适的算法并合理处理噪声，才能发挥聚类最大的商业与科研价值。